

GUÍA COMPLETA — LABORATORIO REMOTO V3

Simulación controlada tipo ransomware con controlador, servidor y dos endpoints.

Después de iniciar el agente en cada víctima, las operaciones de demostración se controlan desde PC 1. Las únicas órdenes remotas son **SIMULAR** y **RECUPERAR**. La simulación no cifra archivos: renombra únicamente archivos ficticios dentro de C:\LAB_RANSOMWARE\Datos.

1. Arquitectura

PC 1 — Controlador/Servidor: 192.168.100.10:5000

PC 2 — Víctima 01: 192.168.100.20

PC 3 — Víctima 02: 192.168.100.30

Flujo: agente → registro → polling → PC1 pone orden → agente ejecuta operación local → agente reporta resultado → PC1 consulta estado.

2. Servidor en PC 1

Crea C:\LAB_RANSOMWARE\Servidor\servidor_lab.py:

```
from flask import Flask, request, jsonify
from datetime import datetime
import json, os, re, threading

HOST = "192.168.100.10"
PORT = 5000
BASE = r"C:\LAB_RANSOMWARE\Servidor"
DB = os.path.join(BASE, "victimas.json")

app = Flask(__name__)
lock = threading.Lock()
pendientes = {}
os.makedirs(BASE, exist_ok=True)

def cargar():
    if not os.path.exists(DB): return {}
    with open(DB, "r", encoding="utf-8") as f: return json.load(f)

def guardar(db):
    tmp = DB + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(db, f, indent=2, ensure_ascii=False)
    os.replace(tmp, DB)

def valido(vid):
    return bool(re.fullmatch(r"LAB-[A-F0-9]{6}", vid or ""))

@app.get("/estado")
def estado():
    return jsonify(cargar())

@app.post("/registro")
def registro():
    d = request.get_json(silent=True) or {}
    vid = d.get("id")
    if not valido(vid): return jsonify(ok=False, error="ID invalido"), 400
    with lock:
        db=cargar(); old=db.get(vid, {})
        db[vid]={ "id":vid, "hostname":d.get("hostname", ""), "ip":d.get("ip", ""),
                  "estado":"CONECTADO", "ultima_conexion":datetime.now().isoformat(timespec="seconds"),
                  "ultimo_resultado":old.get("ultimo_resultado", "") }
        guardar(db)
    return jsonify(ok=True)

@app.post("/poll")
def poll():
    d=request.get_json(silent=True) or {}; vid=d.get("id")
    if not valido(vid): return jsonify(ok=False, error="ID invalido"), 400
    with lock:
        db=cargar()
        if vid not in db: return jsonify(ok=False, error="No registrado"), 404
        db[vid]["estado"]="CONECTADO"
```

```

        db[vid]["ultima_conexion"]=datetime.now().isoformat(timespec="seconds")
        guardar(db); cmd=pendientes.pop(vid,None)
        return jsonify(ok=True,comando=cmd)

@app.post("/ordenar")
def ordenar():
    d=request.get_json(silent=True) or {}; vid=d.get("id"); cmd=d.get("comando")
    if not valido(vid): return jsonify(ok=False,error="ID invalido"),400
    if cmd not in ("SIMULAR","RECUPERAR"):
        return jsonify(ok=False,error="Operacion no permitida"),400
    with lock:
        db=cargar()
        if vid not in db: return jsonify(ok=False,error="No registrado"),404
        pendientes[vid]=cmd
    return jsonify(ok=True,mensaje=f"{cmd} puesta en cola")

@app.post("/resultado")
def resultado():
    d=request.get_json(silent=True) or {}; vid=d.get("id")
    if not valido(vid): return jsonify(ok=False,error="ID invalido"),400
    with lock:
        db=cargar()
        if vid not in db: return jsonify(ok=False,error="No registrado"),404
        db[vid]["estado"]=d.get("estado","FINALIZADO")
        db[vid]["ultimo_resultado"]=d.get("resultado","")
        db[vid]["ultima_conexion"]=datetime.now().isoformat(timespec="seconds")
        guardar(db)
    return jsonify(ok=True)

if __name__=="__main__":
    app.run(host=HOST,port=PORT)

```

Instala y ejecuta:

```

py -m pip install flask requests
py servidor_lab.py

```

Comprueba: <http://192.168.100.10:5000/estado>

3. Controlador en PC 1

Guarda Controlador.py en C:\LAB_RANSOMWARE\Servidor:

```

import requests, os
SERVER="http://192.168.100.10:5000"

def estado():
    return requests.get(SERVER+"/estado",timeout=5).json()

def ordenar(vid,cmd):
    return requests.post(SERVER+"/ordenar",json={"id":vid,"comando":cmd},timeout=5).json()

while True:
    os.system("cls")
    print("="*58)
    print("          CONTROLADOR - LABORATORIO REMOTO")
    print("="*58)
    try:
        db=estado()
        for vid,v in db.items():
            print(f'{vid:12} {v.get("hostname",""):18} {v.get("estado","")}')
    except Exception as e:
        print("Servidor no disponible:",e)

    print("\n[1] Actualizar/listar")
    print("[2] Simular cifrado")
    print("[3] Recuperar")
    print("[4] Ver ultimo resultado")
    print("[5] Salir")
    op=input("\nOpcion: ").strip()

    if op=="1": input("ENTER...")
    elif op in ("2","3"):
        vid=input("ID de victima: ").strip().upper()
        cmd="SIMULAR" if op=="2" else "RECUPERAR"
        try: print(ordenar(vid,cmd))
        except Exception as e: print("Error:",e)
        input("ENTER...")
    elif op=="4":
        vid=input("ID: ").strip().upper()
        print(estado().get(vid,"No encontrada")); input("ENTER...")

```

```
elif op=="5": break
```

Ejecuta en otra consola:

```
py Controlador.py
```

4. Agente para PC 2 y PC 3

Guarda agente_lab.py en una máquina de preparación:

```
import os, json, socket, time, uuid, requests

SERVER="http://192.168.100.10:5000"
BASE=r"C:\LAB_RANSOMWARE"
DATOS=os.path.join(BASE, "Datos")
REG=os.path.join(BASE, "registro_victima.json")
INTERVALO=3
os.makedirs(DATOS, exist_ok=True)

def cargar_id():
    if os.path.exists(REG):
        with open(REG, "r", encoding="utf-8") as f: return json.load(f)["id"]
    vid="LAB-"+uuid.uuid4().hex[:6].upper()
    with open(REG, "w", encoding="utf-8") as f: json.dump({"id":vid}, f, indent=2)
    return vid

def archivos_lab():
    return [os.path.join(root, n) for root, _, files in os.walk(DATOS)
            for n in files if n!="SIMULACION_RANSOMWARE.txt"]

def simular():
    # No cifra ni elimina datos: solo renombra archivos del directorio de laboratorio.
    afectados=[]
    for src in archivos_lab():
        if src.endswith(".simulado"): continue
        dst=src+".simulado"; os.rename(src, dst)
        afectados.append(os.path.basename(dst))
    with open(os.path.join(DATOS, "SIMULACION_RANSOMWARE.txt"), "w", encoding="utf-8") as f:
        f.write("SIMULACION CONTROLADA\n")
        f.write("Archivos renombrados a .simulado; no fueron cifrados.\n")
        f.write("Archivos afectados: "+str(len(afectados))+"\n")
    return f"SIMULACION_COMPLETADA: {len(afectados)} archivos"

def recuperar():
    recuperados=[]
    for src in archivos_lab():
        if src.endswith(".simulado"):
            dst=src[:-9]
            if not os.path.exists(dst): os.rename(src, dst); recuperados.append(os.path.basename(dst))
    marker=os.path.join(DATOS, "SIMULACION_RANSOMWARE.txt")
    if os.path.exists(marker): os.remove(marker)
    return f"RECUPERACION_COMPLETADA: {len(recuperados)} archivos"

def resultado(vid, estado, texto):
    try: requests.post(SERVER+"/resultado", json={"id":vid, "estado":estado, "resultado":texto}, timeout=5)
    except requests.RequestException: pass

def registrar(vid):
    try: requests.post(SERVER+"/registro", json={"id":vid, "hostname":socket.gethostname()}, timeout=5)
    except requests.RequestException: pass

def main():
    vid=cargar_id(); print("ID:", vid); registrar(vid)
    while True:
        try:
            cmd=requests.post(SERVER+"/poll", json={"id":vid}, timeout=5).json().get("comando")
            if cmd=="SIMULAR":
                try:
                    resultado(vid, "EJECUTANDO", "SIMULAR recibida")
                    resultado(vid, "SIMULACION_COMPLETADA", simular())
                except Exception as e: resultado(vid, "ERROR", str(e))
            elif cmd=="RECUPERAR":
                try:
                    resultado(vid, "EJECUTANDO", "RECUPERAR recibida")
                    resultado(vid, "RECUPERADO", recuperar())
                except Exception as e: resultado(vid, "ERROR", str(e))
        except requests.RequestException: pass
        time.sleep(INTERVALO)

if __name__=="__main__": main()
```

Genera el EXE:

```
py -m pip install requests pyinstaller  
pyinstaller --onefile --noconsole agente_lab.py
```

Copia dist\agente_lab.exe a PC 2 y PC 3.

5. Preparar las víctimas

Crea solamente el directorio de laboratorio:

```
C:\LAB_RANSOMWARE\  
■■■ Datos\  
    ■■■ documento1.txt  
    ■■■ documento2.txt  
    ■■■ ejemplo.docx  
    ■■■ ejemplo.pdf
```

Usa archivos ficticios. No apuntes el agente a Documentos, Escritorio, perfiles, unidades de red o información institucional.

Ejecuta agente_lab.exe en cada víctima. Generará un ID LAB-XXXXXX, se registrará y empezará a consultar el servidor cada 3 segundos.

6. Demostración desde PC 1

SIMULAR: Controlador → seleccionar ID → el servidor pone la tarea en cola → el agente la recibe → los archivos de Datos pasan temporalmente a .simulado → se reporta el resultado.

RECUPERAR: desde el mismo Controlador selecciona el ID → el agente recibe RECUPERAR → revierte los nombres → elimina el marcador. No necesitas volver a intervenir manualmente en la víctima.

7. Prueba con las dos víctimas

Ejecuta SIMULAR para LAB-Axxxxx, verifica PC 2; luego para LAB-Bxxxxx, verifica PC 3. Finalmente ejecuta RECUPERAR para cada ID.

8. Estados y telemetría

victimas.json conserva ID, hostname, IP si se suministra, estado, última conexión y último resultado.

9. Pruebas de fallo

Detén el servidor y comprueba que el agente sigue intentando. Reinicia el servidor y verifica la reconexión. Prueba un ID inexistente. Comprueba que una operación distinta de SIMULAR/RECUPERAR es rechazada. Verifica que nada fuera de C:\LAB_RANSOMWARE\Datos cambia.

10. Presentación técnica

Puedes presentar el flujo como un modelo de **tasking remoto controlado**: endpoint → polling → tarea → ejecución local predefinida → telemetría. La restricción deliberada es que no existe un intérprete remoto de comandos ni ejecución de código arbitrario.

11. Solución de problemas

No aparecen víctimas: revisa conectividad hacia 192.168.100.10:5000 y el servidor activo.

No llega la orden: espera el intervalo de 3 segundos y revisa el proceso del agente.

Estado antiguo: revisa ultima_conexion y que el agente continúe ejecutándose.

PyInstaller falla: prueba primero agente_lab.py con Python en la máquina de preparación.

12. Seguridad del laboratorio

Mantén las VMs aisladas, utiliza snapshots y archivos ficticios. No conectes el laboratorio a carpetas compartidas ni datos institucionales. Esta guía no implementa ejecución remota arbitraria, propagación, explotación ni persistencia.